Chapter 6

Support Vector Machines

Support Vector Machines (SVMs) are a very important tool in classification and are widely used in a variety of settings. The support vector machine (SVM) model was first introduced by Vladimir Vapnik (it is described in Vapnik, V., *The Nature of Statistical Learning Theory*, 1996, see Additional Reading). It is largely based on the idea of geometrically solving classification problems using optimisation, so, the flavour of this chapter is less statistical than the previous ones.

Vapnik's main idea was the following. Consider a binary classification problem: **a good way to classify discrete binary samples is to find a (hyper)plane that optimally separates the samples belonging to the two classes**. This objective can be posed as an optimisation problem where our outcome will be the parameters that define the hyperplane. So our target is to split our space into two half-spaces where the majority of class '1' lies in one of the half-spaces (and vice versa for class '0').

We will study the problem of *hard* classification of binary samples via the SVM. (Recall that by *hard* classification we mean i.e. a strict, non-probabilistic assignment of samples to one class, in opposition to *soft* classification, where we assign, to each sample, a probability of belonging to each class.

The SVM is a *linear* classification method, like logistic regression (Chapter 3) - this should come as no surprise as we look for a *separating hyperplane*.We will discuss later in the chapter how to extend these ideas to *soft* (i.e., probabilistic) classification, as well as to *nonlinear* classifiers.

> *Quick Aside.* SVMs are naturally binary classifiers (i.e., for two-class outcomes). With some considerable extra work (e.g. one-to-one and one-to-all strategies) that would exceed the scope of this course, they can be generalised to multi-class problems as well, but we will stick here to binary classifiers.

1. Formulation of the problem: Hard-margin SVMs

We start with our familiar setup, where the input variables are continuous and the output variable is binary:

$$\mathbf{x}^{(i)} = (x_1^{(i)}, \ldots, x_p^{(i)}) \in \mathbb{R}^p \quad \rightarrow \quad y^{(i)} \in \{-1, +1\}, \quad i = 1, \ldots, N$$

Without loss of generality, we have chosen -1 and +1 as our outcome 'classes' to make the mathematical derivations further down a little easier and more compact, as will become clear immediately.

To grasp the general concept visually, we consider the case where $\mathbf{x}^{(i)} \in \mathbb{R}^2$, i.e. $\mathbf{x}^{(i)} = (x_1^{(i)}, x_2^{(i)})$, which can be easily drawn as a sketch (Figure 6.1). As described above, the goal of SMV's is intuitively easy: find the hyperplane in our input space that separates one class from the other (for $\mathbf{x} \in \mathbb{R}^2$ the hyperplane is a line; for $\mathbf{x} \in \mathbb{R}^3$, the hyperplane is a plane). The hyperplane gives the "decision boundary" of this classification problem, that is, the boundary that allows us to decide to what class each data point belongs to.

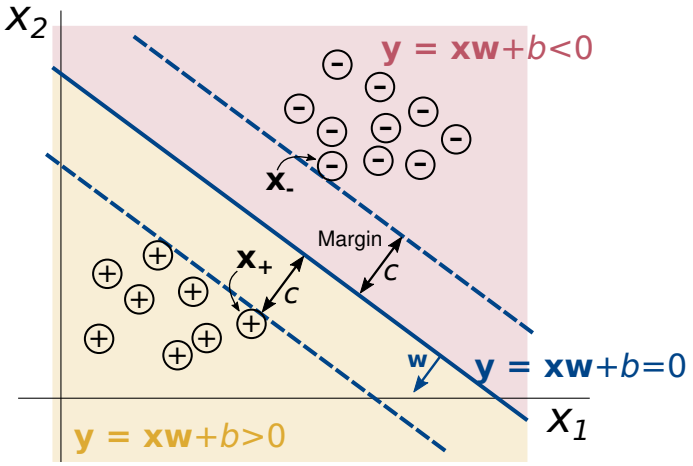


Figure 6.1. Sketch of an SVM with two predictors and a binary outcome. The separating hyperplane (here a line) is given in blue, each of its sides (coloured in either red or yellow) corresponds to a predicted class.

For $\mathbf{x} \in \mathbb{R}^2$, the hyperplane we are looking for is a line parameterised as:

$$x_2 = wx_1 + b$$

This can be written in vector notation as:

$$\mathbf{x} \cdot \mathbf{w} + b = 0$$
$$\mathbf{w} = \begin{pmatrix} w \\ -1 \end{pmatrix}$$

where $\mathbf{w}$ is the vector normal to the line (see Figure 6.1). Of course, this can be generalised to p dimensions directly. For $\mathbf{x} \in \mathbb{R}^p$, our hyperplane will be given by:

$$\mathbf{x} \cdot \mathbf{w} + b = 0$$
$$\mathbf{w} = \begin{pmatrix} w_1 \\ \vdots \\ w_p \end{pmatrix}$$

and $\mathbf{w}$ is again the vector normal to the hyperplane.

Our goal is to obtain $\mathbf{w}$ and b that parametrise our hyperplane. Once we have determined both, we can use the following rule:

Key message: the prediction by a **SVM classifier** is:

$$\text{Given } \mathbf{x}^{\text{in}}, \text{ if } \begin{cases} \mathbf{x}^{\text{in}} \cdot \mathbf{w} + b \geq 0 & \text{then } \hat{y} = +1 \\ \mathbf{x}^{\text{in}} \cdot \mathbf{w} + b < 0 & \text{then } \hat{y} = -1 \end{cases} \tag{6.1}$$

because the SVM decision boundary is the hyperplane $\mathbf{x} \cdot \mathbf{w} + b = 0$.

This corresponds to the point $\mathbf{x}^{\text{in}}$ falling in the yellow or red region, respectively, in Figure 6.1.

As can be seen in Figure 6.1, there could be an infinite number of lines that would separate the data. Intuitively, the optimally separating line is here meant as the one that allows for the 'widest street' possible through the data. SVMs approach this problem through the concept of *margin*, which denotes the smallest distance between the decision boundary (the line in the simple example of Figure 6.1) and any of the samples. Maximising the margin is what gives of the 'widest street' between data points, hence, **in SVMs, the decision boundary is chosen to be the one maximising the margin in any direction**.

Let's look at this in a few steps focussing on the example of Figure 6.1. First of all, the line that is maximally far away from both sets of data points is half-way in-between, hence we have used the same symbol c to denote the magnitude of the margin on both sides. Let $\mathbf{x}_+$ be the closest data point of class +1 to the dividing line and similarly for $\mathbf{x}_-$ for class -1 (Figure 6.1). Then we have the following:

$$\begin{cases} \mathbf{x}_+ \cdot \mathbf{w} + b = c \\ \mathbf{x}_- \cdot \mathbf{w} + b = -c \end{cases} \tag{6.2}$$

Geometrically, the width of the 'street' between points is given by:

$$\text{width} = (\mathbf{x}_+ - \mathbf{x}_-) \cdot \frac{\mathbf{w}}{\|\mathbf{w}\|}$$

and it follows immediately from (6.2) that

$$\text{width} = \frac{2c}{\|\mathbf{w}\|}$$

Note that c is arbitrary through re-scaling of the data. Therefore, it can be fixed to be unity:

$$c := 1$$

and we finally have:

$$\text{width} = \frac{2}{\|\mathbf{w}\|} \tag{6.3}$$

This expression tells us that in order to maximise the width (and hence the margin), we have to minimise $\|\mathbf{w}\|$.

We call the **margin constraint** the condition that the margin should lie entirely between training data points, or, in other words, that training data points should be outside of (or, at most on) the margin. Mathematically:

$$(\mathbf{x}^{(i)} \cdot \mathbf{w} + b) \geq 1 \quad \text{if} \quad \mathbf{x}^{(i)} \in \{1\}, \qquad (\mathbf{x}^{(i)} \cdot \mathbf{w} + b) \leq -1 \quad \text{if} \quad \mathbf{x}^{(i)} \in \{-1\}$$

Our choice of notation for the outcome classes as $\{-1, +1\}$ now comes in handy, as it allows us to write the margin constraint compactly as:

$$y^{(i)}(\mathbf{x}^{(i)} \cdot \mathbf{w} + b) \geq 1$$

This, combined with (6.3), allows us to obtain:

Key message: the hard classification SVM optimisation problem is:

$$\min_{\mathbf{w}} \frac{1}{2}\|\mathbf{w}\|^2 \quad \text{subject to} \quad y^{(i)}(\mathbf{x}^{(i)} \cdot \mathbf{w} + b) \geq 1, \qquad i = 1, \dots, N \tag{6.4}$$

In words, we are looking to minimise $\|\mathbf{w}\|$ such that all points $\mathbf{x}^{(i)}$ are above or below (depending on their class) the dividing line $\mathbf{x}^{(i)} \cdot \mathbf{w} + b = 0$ and at a distance of at least 1 (which is the distance to the hyperplane from both $\mathbf{x}_+$ and $\mathbf{x}_-$).

Quick Aside: The solution of the optimisation problem (6.4) is not analytical, but can be obtained via numerical techniques of convex optimisation that we will not cover systematically in this course. A few pointers to **Quadratic Programming** are however important.

Quadratic programming is used when the objective is to optimise a *non-linear* (e.g. quadratic) function with respect to *linear* inequality constraints:

$$\min_{\mathbf{z}} f(\mathbf{z}) \text{ subject to } g_i(\mathbf{z}) \leq 0 \text{ for } i = 1, \dots, m$$

This is our problem (6.4), where the optimisation variables $\mathbf{z}$ are $\mathbf{w}$ and b.

The solution to this problem is given by the Karush-Kuhn-Tucker (KKT) conditions, derived first in 1939 and, in more general form, in 1951. Briefly, we construct a Lagrangian function with Lagrange multipliers $\alpha_i \geq 0$:

$$\mathcal{L}(\mathbf{z}; \boldsymbol{\alpha}) = f(\mathbf{z}) + \sum_{i=1}^{m} \alpha_i g_i(\mathbf{z})$$

Based on the KKT conditions (whose proof is beyond the scope of this course), one has that, at the minimum $\mathbf{z}^\star$, there exists

$$\boldsymbol{\alpha} = (\alpha_1, \dots, \alpha_m)$$

such that:

$$\nabla f(\mathbf{z}^\star) + \sum_{i=1}^{m} \alpha_i \nabla g_i(\mathbf{z}^\star) = 0 \quad \text{(Lagrangian vanishes)}$$

$$g_i(\mathbf{z}^\star) \leq 0 \qquad \forall i \quad \text{(primal costraints)}$$

$$\alpha_i \geq 0 \qquad \forall i \quad \text{(dual constraints)}$$

$$(\spadesuit) \qquad \alpha_i \cdot g_i(\mathbf{z}^\star) = 0 \qquad \forall i \quad \text{(complementary slackness conditions)}$$

Without going into the details of this optimisation, the α_i are found by solving the *dual* problem (hence the name "dual" constraints), which is:

$$\max_{\boldsymbol{\alpha}} \left\{ \inf_{\mathbf{z}} \mathcal{L}(\mathbf{z}; \boldsymbol{\alpha}) \right\} \quad \text{subject to} \quad \alpha_i \geq 0, \forall i$$

associated to our original minimisation task (the *primal* problem). The *duality principle* states that an optimisation be formulated in two specular ways – the primal and dual formulation. In short, if the primal problem is a minimisation (like here), the dual problem is a maximisation, and viceversa (see again the book *Convex Optimisation* by Stephen Boyd for an exhaustive discussion).

As noted in the aside, one can resort to quadratic programming since we have a non-linear objective function with linear constraints. To fit the notation from the aside, we can now rewrite the problem slightly:

$$\min_{\mathbf{w}} \frac{1}{2}\|\mathbf{w}\|^2 \text{ subject to } \underbrace{1 - y^{(i)}(\mathbf{x}^{(i)} \cdot \mathbf{w} + b)}_{g_i(\mathbf{w}, b)} \leq 0 \text{ for } i = 1, \dots, N$$

and we construct the Lagrangian:

$$\mathcal{L}(\mathbf{w}, b; \boldsymbol{\alpha}) = \frac{1}{2}\|\mathbf{w}\|^2 + \sum_{i=1}^{N} \alpha_i \big(1 - y^{(i)}(\mathbf{x}^{(i)} \cdot \mathbf{w} + b)\big)$$

As seen in the aside, we take the gradient of the Lagrangian:

$$\left\{ \begin{array}{l} \dfrac{\partial \mathcal{L}}{\partial b} = -\displaystyle\sum_{i=1}^{N} y^{(i)} \alpha_i \\ \nabla_{\mathbf{w}} \mathcal{L} = \mathbf{w} - \displaystyle\sum_{i=1}^{N} \alpha_i y^{(i)} \mathbf{x}^{(i)} \end{array} \right\}$$

At the minimum, the equations above equal to 0 and so we get:

$$\left\{ \begin{array}{l} \displaystyle\sum_{i=1}^{N} y^{(i)} \alpha_i = 0 \\ \mathbf{w}^\star = \displaystyle\sum_{i=1}^{N} \alpha_i y^{(i)} \mathbf{x}^{(i)} \end{array} \right\}$$

Now we expand and rewrite $\mathcal{L}$ in terms of $\boldsymbol{\alpha}$ only:

$$\mathcal{L}(\boldsymbol{\alpha}) = \frac{1}{2}\mathbf{w} \cdot \mathbf{w} + \sum_{i=1}^{N} \alpha_i - \underbrace{\sum_{i=1}^{N} \alpha_i y^{(i)} \mathbf{x}^{(i)}}_{=\mathbf{w}} \cdot \mathbf{w} - \underbrace{\sum_{i=1}^{N} \alpha_i y^{(i)} b}_{=0}$$

And then we substitute the results from before (as can be seen from the underbraces):

$$\begin{aligned} \mathcal{L}(\boldsymbol{\alpha}) &= \frac{1}{2}\mathbf{w} \cdot \mathbf{w} + \sum_{i=1}^{N} \alpha_i - \mathbf{w} \cdot \mathbf{w} = \sum_{i=1}^{N} \alpha_i - \frac{1}{2}\mathbf{w} \cdot \mathbf{w} \\ &= \sum_{i=1}^{N} \alpha_i - \frac{1}{2} \sum_{i,j=1}^{N} \alpha_i \alpha_j y^{(i)} y^{(j)} \mathbf{x}^{(i)} \cdot \mathbf{x}^{(j)} \end{aligned} \tag{6.5}$$

Here, the last equation was obtained by substituting $\mathbf{w}^\star = \sum_{i=1}^{N} \alpha_i y^{(i)} \mathbf{x}^{(i)}$ into it. This expression shows that the Lagrangian can be expressed only in terms of the samples $\{\mathbf{x}^{(i)}, y^{(i)}\}$ and the Lagrange multipliers α_i, which are found (see aside) by solving the associated dual problem:

$$\max_{\boldsymbol{\alpha}} \left\{ \inf_{\mathbf{w}, b} \mathcal{L}(\mathbf{w}, b; \boldsymbol{\alpha}) \right\} \quad \text{subject to} \quad \alpha_i \geq 0, \, \forall i$$

To get to the key point of Vapnik's SVM result, we look back at the aside above, and specifically at the complementary slackness conditions (♠) that appear from the KKT conditions. They imply that either the primal constraint is tight or the dual constraint is tight—one of them has to be strictly zero for the complementary slackness conditions to be verified. From that equation it then follows that:

$$\alpha_i = 0 \text{ if } \mathbf{x}^{(i)} \neq \mathbf{x}_+ \text{ or } \mathbf{x}_-$$

In words: *the Lagrange multipliers are only present to enforce the constraint at the minimal points* $\mathbf{x}_+$ *and* $\mathbf{x}_-$, that is, we recover the construction (6.2):

$$\begin{cases} \mathbf{x}_+ \cdot \mathbf{w} + b = 1 \\ \mathbf{x}_- \cdot \mathbf{w} + b = -1 \end{cases}$$

which can be used to estimate b. Putting together these different results, we obtain:

Key message: Only the two minimally distant points $\mathbf{x}_+$ and $\mathbf{x}_-$ define the vector $\mathbf{w}^\star$ and b of the margin-maximising hyperplane of a hard-margin SVM, that is:

$$\mathbf{w}^\star = \sum_{i=1}^{N} \alpha_i \, y^{(i)} \mathbf{x}^{(i)} = \alpha_+ \mathbf{x}_+ - \alpha_- \mathbf{x}_-$$

$$b = 1 - \mathbf{x}_+ \cdot \mathbf{w}^\star = 1 - \mathbf{x}_+ \cdot [\alpha_+ \mathbf{x}_+ - \alpha_- \mathbf{x}_-] = 1 - \alpha_+ \mathbf{x}_+ \cdot \mathbf{x}_+ + \alpha_- \mathbf{x}_+ \cdot \mathbf{x}_-$$

where α_+ and α_- are determined via the dual-problem formulation. The two points $\mathbf{x}_+$ and $\mathbf{x}_-$ are called the *support vectors* of the hyperplane (hence the name SVM).

Once we have obtained the parameters of the hyperplane, we can use the model to classify any new sample using the classifier (6.1).

2. Soft-margin SVMs

In this section, we will now look at ways to extend SVMs to more realistic scenarios. Indeed, the hard-margin SVM as formulated above requires the data points to be separable in p-dimensional space, but this is not possible in most applications. When the separating hyperplane does not exist, we say the hard-margin classification SVM problem is infeasible. Our formulation above contains the seeds of how to extend the method naturally to these cases when a strictly separating hyperplane does not exist, i.e. the case of *imperfect separation*. In short, **to deal with imperfect separation, we relax the optimisation by introducing a *penalty* term in the loss function (soft-margin SVM)**.

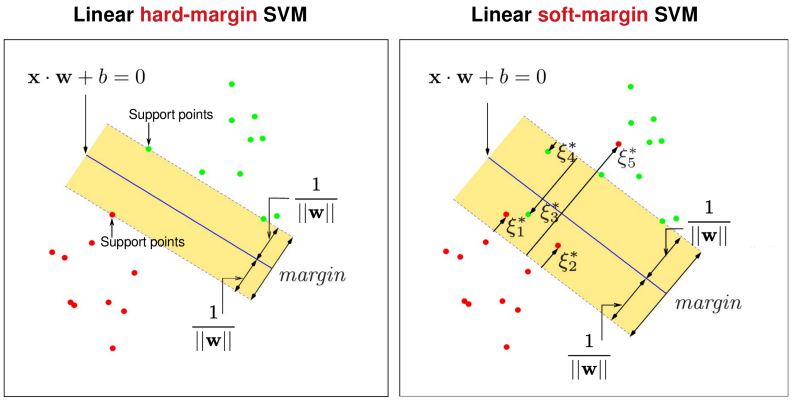


Figure 6.2. In the case of imperfect separation (right), contrary to the linearly separable case (left), one learns the hyperplane allowing some data points (here marked by ξ_j^*) to be on the wrong side of the margin (by an amount given by ξ_j^* itself). Figure adapted from Hastie, Tibshirani, Friedman, *The Elements of Statistical Learning*, Chapter 12.

As discussed above, the margin constraint for a given data point $\mathbf{x}^{(i)}$ is $y^{(i)}(\mathbf{x}^{(i)}\mathbf{w} + b) \geq 1$. Since it's impossible to find a margin-maximising hyperplane such that all the data points respect this constraint, in the soft-margin SVM optimisation one adopts a *relaxed* version of the margin constraint:

$$y^{(i)}(\mathbf{x}^{(i)} \cdot \mathbf{w} + b) \geq 1 - \xi_i \quad \text{or, equivalently} \quad 1 - y^{(i)}(\mathbf{x}^{(i)} \cdot \mathbf{w} + b) \leq \xi_i$$

where ξ_i represents violations of the hard margin constraint. In other words, one learns a hyperplane that still tries to maximise the margin, but also allows for points to violate

the margin constraint (i.e., to be on the wrong side of the margin, see Figure 6.2), while penalising the magnitude of this violation.

To do this, one models the violations through the *hinge loss*, defined as follows:

$$\xi_i = \max\left\{0,\ \underbrace{1 - (\mathbf{x}^{(i)} \cdot \mathbf{w} + b)y^{(i)}}_{\text{size of violation}}\right\} \tag{6.6}$$

such that $\xi_i \geq 0$ by construction (see Figure 6.3). More specifically $\xi_i \neq 0$ only for the points that violate the margin constraint, with its value reflecting the size of the violation (Figure 6.2).

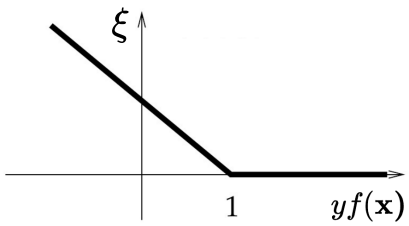


Figure 6.3. Hinge loss ξ as a function of $yf(\mathbf{x})$, where $f(\mathbf{x}) = \mathbf{x}\cdot\mathbf{w}+b$. The soft-margin optimisation is defined in such a way as either to keep ξ at zero (for $yf(\mathbf{x}) > 1$, i.e., where the hard-margin constraint is satisfied because $\mathbf{x}$ is on the correct size of the margin) or to keep the size of the violation small (for $yf(\mathbf{x}) < 1$).

Since $\sum_{i=1}^{N} \xi_i$ measures the total magnitude of the margin constraint violation, the idea is to add this hinge term to the loss in such a way as to learn the parameters of the hyperplane while keeping the magnitude of the violation small.

Key message: The optimisation to learn the soft-margin SVM is:

$$\min_{\mathbf{w},b}\left(\frac{1}{2}\|\mathbf{w}\|^2 + \lambda\sum_{i=1}^{N}\xi_i\right) \text{ subject to } 1 - y^{(i)}(\mathbf{x}^{(i)}\cdot\mathbf{w}+b) \leq \xi_i \text{ for } i = 1,\ldots,N$$

where ξ_i is the hinge loss (Equation 6.6, Figure 6.3) and serves to penalise margin constraint violations. The hyperparameter λ controls how 'hard' the margin constraint can be (hardness), allowing for larger or smaller violations.

This hyperparameter can be tuned by various methods such as cross-validation. In the limit of large λ, the soft-margin SVM optimisation approaches the hard-margin SVM.

Following this formulation, the same optimisation procedure relying on the dual problem described in Section 1 can be applied, simply with the additional hyperparameter λ. Its details are beyond the scope of this course, but you can read about them in Hastie, Tibshirani, Friedman, *The Elements of Statistical Learning*, Chapter 12, Section 12.2.1. In particular, from the KKT conditions in this case one finds that only for the data points $\mathbf{x}^{(i)}$ for which the relaxed margin constraint is exactly met:

$$y^{(i)}(\mathbf{x}^{(i)}\cdot\mathbf{w}+b) = 1 - \xi_i$$

the Lagrange multipliers are non-zero, i.e., these data points are support vectors. Thus, in the soft-margin SVM there are support vectors such that:

- $\xi_i = 0$: Support vectors on the boundary of the margin (correctly classified);
- $0 < \xi_i < 1$: Support vectors that are within the margin, and are correctly classified;
- $\xi_i > 1$: Support vectors that are within the margin, but are misclassified.

2.1. Training SVMs: gradient descent and minibatch SGD. To train an SVM, both in the hard and soft-margin version, we need to solve an optimisation problem. For instance, in the soft-margin SVM we find the optimal To train an SVM, both in the hard and soft-margin version, we need to solve an optimisation problem. For instance, in the soft-margin SVM we find the optimal $\mathbf{w}$ and b by minimising the loss function:

(6.7)
$$L(\mathbf{w}, b; \mathcal{S}) := \left(\frac{1}{2}\|\mathbf{w}\|^2 + \lambda \sum_{i=1}^{|\mathcal{S}|} \xi_i\right) \text{ subject to } 1 - y^{(i)}(\mathbf{x}^{(i)} \cdot \mathbf{w} + b) \leq \xi_i \text{ for } \{\mathbf{x}^{(i)}, y^{(i)}\} \in \mathcal{S}$$

where we have denoted by $\mathcal{S} = \{\mathbf{x}^{(i)}, y^{(i)}\}$ the training set, of size $|\mathcal{S}| = N^{\text{training}}$. Let's denote in a compact way the model parameters as $\boldsymbol{\theta} = \{\mathbf{w}, b\}$. As we have seen in Chapter 1, the gradient $\nabla_{\boldsymbol{\theta}} L(\boldsymbol{\theta}_0; \mathcal{S})$ evaluated at $\boldsymbol{\theta}_0$ defines the direction of steepest ascent in parameter space at the point $\boldsymbol{\theta}_0$. The gradient descent algorithm takes an initial guess for the parameters $\boldsymbol{\theta}_0$ and updates, at each iteration t, the parameter values according to the rule:

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta_t \nabla_{\boldsymbol{\theta}} L(\boldsymbol{\theta}_t; \mathcal{S})$$

where $\eta_t > 0$ is a small learning rate which may depend on t. For a suitably chosen η_t, the iterations converge to a minimum for $\boldsymbol{\theta}$ - specifically, a *global* minimum for linear soft-margin SVM, as the loss is convex.

However, computing $\nabla_{\boldsymbol{\theta}} L(\boldsymbol{\theta}_t; \mathcal{S})$ as above requires computing the gradients of the per-example loss for every element in the training set. For large data sets (and large models) this can be prohibitively expensive. Minibatch stochastic gradient descent (minibatch SGD) provides a cheaper estimate of the full gradient, by computing the gradient on a minibatch of data points, instead of the full data set.

> **Key message:** In minibatch SGD, the parameter update rule is:
>
> (6.8)
> $$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta_t \nabla_{\boldsymbol{\theta}} L(\boldsymbol{\theta}_t; \mathcal{S}_m)$$
>
> where $L(\boldsymbol{\theta}_t; \mathcal{S}_m)$ is the mean loss over $\mathcal{S}_m$, a randomly drawn sample (minibatch) of training data points, of size $|\mathcal{S}_m|$ typically much smaller than $|\mathcal{S}|$.

This update provides a stochastic approximation to the full-dataset gradient that is far more efficient to compute, and provides a huge speed up in the training process for large data sets, while still converging to the global minimum.

One can even use minibatch size $|\mathcal{S}_m| = 1$, i.e., consisting of just one data point at a time - this version is simply referred to as SGD.

You will see the implementation of minibatch SGD in the coding tasks for this week.

3. Beyond linear classification: Kernelised SVMs

The second important extension of SVMs is to consider non-linear classification, suitable for non-linearly separable problems. So far, the goal has been to find a *linear* hyper-plane to separate the data (hardly or softly). If the data is not linearly separable (even softly), then such methods will not work well (i.e., there will not be a 'good' hyperplane that can separate the classes).

To extend the method, we invoke *kernel* functions, which bring the following key advantage: **they allow one to map non-linearly separable feature space into a higher-dimensional space (the kernelised feature space) where data become linearly separable**.

3.1. Kernel functions. A kernel function is an analogue of a dot (or inner) product. In general, a kernel function maps a pair of vectors in a lower dimensional space (e.g., $\mathbb{R}^d$) to give the dot product of non-linearly transformed vectors in a higher-dimensional space ($\mathbb{R}^D$). This is a function of the following form:

$$k(\mathbf{x}, \mathbf{y}) = \boldsymbol{\phi}(\mathbf{x}) \cdot \boldsymbol{\phi}(\mathbf{y}) \tag{6.9}$$

where $\mathbf{x}, \mathbf{y} \in \mathbb{R}^d$ (vectors of size d) and $\boldsymbol{\phi}(\mathbf{x}), \boldsymbol{\phi}(\mathbf{y}) \in \mathbb{R}^D$. The function k is a kernel with associated $\boldsymbol{\phi}$ if and only if k is positive semi-definite.

> *Quick Aside.* The theory of kernels is rich and very important both in mathematics and in machine learning, but in this course we only use them sparingly to study the non-linear extensions of SVMs. For further reading on kernels in relation to SVMs and machine learning generally, see Hastie, Tibshirani, Friedman, *The Elements of Statistical Learning*, Chapters 12, 5, and 14.

Some widely used kernels in applications are:

- Polynomial (of degree up to n):
$$k(\mathbf{x}, \mathbf{y}) = (\mathbf{x} \cdot \mathbf{y} + c)^n$$
- Polynomial of degree n:
$$k(\mathbf{x}, \mathbf{y}) = (\mathbf{x} \cdot \mathbf{y})^n$$
- Gaussian kernel (or radial basis function kernel):
$$k(\mathbf{x}, \mathbf{y}) = e^{-\frac{\|\mathbf{x}-\mathbf{y}\|^2}{\sigma}}$$
- Sigmoid:
$$k(\mathbf{x}, \mathbf{y}) = \tanh\big(\beta(\mathbf{x} \cdot \mathbf{y}) + c\big)$$

Starting from these standard kernels, one can also construct new kernels by combining them according to sets of well-defined rules - see chapter 6.2 of Bishop, *Pattern Recognition and Machine Learning* - enabling additional flexibility in the kernel choice.

3.2. The kernel trick. Kernels are interesting for the following property, which we illustrate with an example. Consider the following:

$$\text{Given} \quad \mathbf{x} = (x_1, x_2) \in \mathbb{R}^2$$
$$\text{define:} \quad \boldsymbol{\phi}(\mathbf{x}) = \begin{pmatrix} x_1^2 \\ x_2^2 \\ \sqrt{2}x_1x_2 \end{pmatrix} \in \mathbb{R}^3$$

Note that we have the following dot product:

$$\boldsymbol{\phi}(\mathbf{x}) \cdot \boldsymbol{\phi}(\mathbf{y}) = x_1^2y_1^2 + x_2^2y_2^2 + 2x_1x_2y_1y_2$$

We can then (cleverly) define a function k such that:

$$\begin{aligned} k(\mathbf{x}, \mathbf{y}) = k(\mathbf{x} \cdot \mathbf{y}) &:= (\mathbf{x} \cdot \mathbf{y})^2 = (x_1y_1 + x_2y_2)^2 \\ &= x_1^2y_1^2 + x_2^2y_2^2 + 2x_1x_2y_1y_2 \\ &= \boldsymbol{\phi}(\mathbf{x}) \cdot \boldsymbol{\phi}(\mathbf{y}) \end{aligned}$$

Hence, we do not need to compute the non-linear mapping $\boldsymbol{\phi}(\mathbf{x})$ to effectively lift our data vectors $\mathbf{x}$ to a D-dimensional space, *if* $\mathbf{x}$ *enters our framework only through pairwise dot products*, which allow us to manipulate them only implicitly. All we need to do is compute

the usually simpler dot product $\mathbf{x} \cdot \mathbf{y}$ and apply the kernel function k associated to $\boldsymbol{\phi}(\mathbf{x})$ by (6.9). Whilst this toy example was simply going from $d = 2$ to $D = 3$ dimensions (not great savings), we could have defined a kernel function in $D = 1000$ dimensions, which much greater savings.

> **Kernel trick** (or *kernel substitution*): an algorithm defined for d-dimensional vectors that can be formulated only in terms of pairwise dot products can be applied to non-linearly transformed, D-dimensional vectors (with $D >> d$ and potentially infinite) by replacing each dot product by a kernel function.

The kernel trick actually fits neatly into the SVM formulation, because all the expressions in our derivation of the optimisation contained 'dot products of vectors'. Let's point out the places where the kernel functions could be included in lieu of the dot products. We have the following model for the hard-margin SVM:

$$\mathbf{w}^\star = \alpha_+ \mathbf{x}_+ - \alpha_- \mathbf{x}_-$$

$$b = 1 - \alpha_+ \underbrace{\mathbf{x}_+ \cdot \mathbf{x}_+}_{\bigstar} + \alpha_- \underbrace{\mathbf{x}_+ \cdot \mathbf{x}_-}_{\bigstar}$$

where the coefficients α_i are found by maximising the Lagrangian (6.5):

$$\mathcal{L}(\boldsymbol{\alpha}) = \sum_{i=1}^{N} \alpha_i - \frac{1}{2} \sum_{i,j=1}^{N} \alpha_i \alpha_j y^{(i)} y^{(j)} \underbrace{\mathbf{x}^{(i)} \cdot \mathbf{x}^{(j)}}_{\bigstar}$$

Furthermore, the soft-margin SVM is trained by minimising the loss function:

$$\frac{1}{2} \underbrace{\mathbf{w} \cdot \mathbf{w}}_{\bigstar} + \lambda \sum_{i=1}^{N} \max \left\{ 0, 1 - y^{(i)} (\underbrace{\mathbf{x}^{(i)} \cdot \mathbf{w}}_{\bigstar} + b) \right\}$$

Finally, given a new data point $\mathbf{x}^{\text{in}}$, the classifier (6.1) is:

$$\underbrace{\mathbf{x}^{\text{in}} \cdot \mathbf{w}^\star}_{\bigstar} + b \geq 0 \implies \hat{y} = +1$$

$$\underbrace{\mathbf{x}^{\text{in}} \cdot \mathbf{w}^\star}_{\bigstar} + b \leq 0 \implies \hat{y} = -1$$

From all the starred ★ bits, we note that not only the SVM optimisation solution, but also the SVM prediction boils down to computing dot products. This is hence where we will use kernel functions to accommodate non-linear data.

3.3. Training a kernelised SVM. In the case of SVMs, we choose an appropriate kernel and train the corresponding SVM exploiting the kernel trick. To see more precisely how this optimisation is carried out, let us introduce the notation $\mathbf{w}_{\boldsymbol{\phi}}$ for the parameters of the SVM with kernel $k(\mathbf{x}, \mathbf{y})$ associated to $\boldsymbol{\phi}(\mathbf{x})$. The optimisation of the kernelised hard-margin SVM would then be written as:

$$\min_{\mathbf{w}_{\boldsymbol{\phi}}} \frac{1}{2} \|\mathbf{w}_{\boldsymbol{\phi}}\|^2 \text{ subject to } 1 - y^{(i)} (\boldsymbol{\phi}(\mathbf{x}^{(i)}) \cdot \mathbf{w}_{\boldsymbol{\phi}} + b) \leq 0 \text{ for } i = 1, \ldots, N \tag{6.10}$$

We introduce a vector $\mathbf{u}$ of size $N \times 1$ such that:

$$\mathbf{w}_{\boldsymbol{\phi}} = \sum_{i=1}^{N} \boldsymbol{\phi}(\mathbf{x}^{(i)}) u_i$$

If we consider the (transpose) full data matrix $\mathbf{X}^T$ (of size $p \times N$) having as columns the data points $\mathbf{x}^{(i)}$, the introduction of the vector $\mathbf{u}$ allows us to write, first:

$$\mathbf{w}_{\boldsymbol{\phi}} \cdot \boldsymbol{\phi}(\mathbf{X}^T) = \mathbf{u}^{\mathrm{T}}\mathbf{K}$$

where $\boldsymbol{\phi}(\mathbf{X}^T)$ (of size $D \times N$) gives the mapping of the data matrix onto the D transformed space. Next we have:

$$\mathbf{w}_{\boldsymbol{\phi}} \cdot \mathbf{w}_{\boldsymbol{\phi}} = \mathbf{u}^{\mathrm{T}}\mathbf{K}\mathbf{u}$$

where the $N \times N$ matrix $\mathbf{K}$ (also called Gram matrix) is defined as:

$$\mathbf{K}_{ij} = \boldsymbol{\phi}(\mathbf{x}^{(i)}) \cdot \boldsymbol{\phi}(\mathbf{x}^{(j)}) \equiv k(\mathbf{x}^{(i)}, \mathbf{x}^{(j)})$$

(the Gram matrix has to be positive semi-definite). As a result of these substitutions we obtain, directly for the soft-margin kernelised SVM:

Key message: The optimisation to learn a kernelised soft-margin SVM can be formulated as:

$$\min_{\mathbf{u},b} \left(\frac{1}{2}\mathbf{u}^{\mathrm{T}}\mathbf{K}\mathbf{u} + \lambda \sum_{i=1}^{N} \xi_i \right) \text{ subject to } 1 - y^{(i)}(\mathbf{K}^{(i)}\mathbf{u} + b) \leq \xi_i \text{ for } i = 1, \ldots, N \tag{6.11}$$

where $\mathbf{K}^{(i)}$ denotes row i of the Gram matrix $\mathbf{K}$, and the hinge loss reads:

$$\xi_i = \max\left\{0, 1 - y^{(i)}(\mathbf{K}^{(i)}\mathbf{u} + b)\right\}$$

Note here the advantage of the kernel trick: the dimension of the optimisation problem in (6.11) is N (the size of the training set), hence we don't need to specify the dimensions of $\mathbf{w}_{\boldsymbol{\phi}}$, nor to work explicitly with an optimisation of such dimension (as it would be the case instead with Equation 6.10), leaving the freedom to introduce potentially very high-dimensional transformations $\boldsymbol{\phi}(\mathbf{x})$.

Quick Aside: The kernel trick can be even more conveniently applied to the dual problem - see again Equation (6.5) - and this is often mentioned as a motivation for working with the dual formulation. In many textbooks and tutorials, you will find descriptions of kernelised SVMs based on the solution of the dual problem via quadratic optimisation since the dual form has the advantages of being, in general, faster and more computationally efficient for large datasets (due to the sparsity of the final solution). In the coding task of this course, we instead focus on the primal formulation (Equation 6.11), which can be solved by gradient-descent methods.

3.4. Why use kernelised SVMs? The idea is that non-linearity and high dimensionality can help with separability and thus, classification. For data points $\mathbf{x}, \mathbf{y}$ in $\mathbb{R}^p$, the kernel trick allows us to work with $\boldsymbol{\phi}(\mathbf{x}) \in \mathbb{R}^D$, where $D \gg p$ potentially. Choosing the $\boldsymbol{\phi}$ with the right properties will, in principle, allow us to separate (classify) the data using a hyperplane in the lifted (higher dimensional) space.

An intuition of why the kernel trick can work is given in Figure 6.4 with an example. There we see a set of binary (red, blue) samples $\mathbf{x} = (x_1, x_2) \in \mathbb{R}^2$ that we need to separate. Figure 6.4(a) shows that there is no 'good' hyperplane (line) in $\mathbb{R}^2$ that will separate the blue and red classes with low violation penalties. However, it is also clear that if we lifted our samples to $\mathbb{R}^3$ in a clever way, we would be able to separate the classes using a plane (i.e., there is a good hyperplane in $\mathbb{R}^3$). This is shown in Figure 6.4(b), where one applies the mapping $\phi(\mathbf{x}) = (x_1, x_2, x_3)$ with $x_3 = x_1^2 + x_2^2$: in the transformed space, the blue and red samples can now be separated with a plane normal to x_3. (A kernel corresponding to this transformation can be constructed via polynomial kernels.) Hence, by lifting our

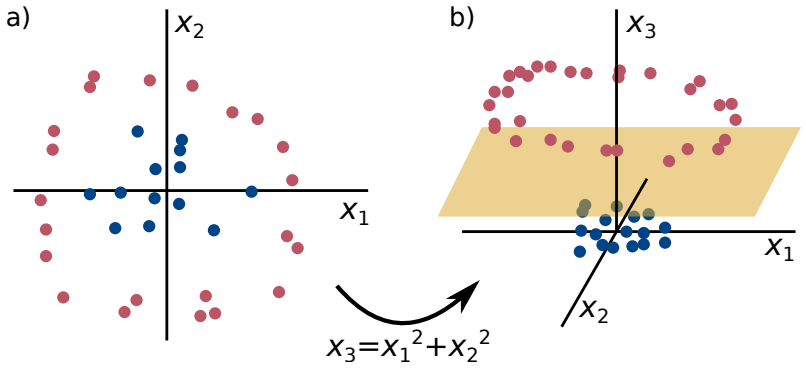


Figure 6.4. Cartoon drawing of the kernel trick. a) shows a set of binary samples that are linearly non-separable in $\mathbb{R}^2$. b) shows the kernel trick in action. Through the addition of a further dimension to the data $x_3 = x_1^2 + x_2^2$, we can make them separable with a (hyper)plane in $\mathbb{R}^3$.

data to a higher dimensional space, we were able to obtain an "easier" problem and use a separating hyperplane for classification.

This principle makes kernel SVMs very successful in applications, and thus widely used. There are typical kernels (including the ones we listed above: polynomial, radial basis functions, sigmoid kernels, etc.) which are standard choices. In many cases, these kernels can deal with nonlinearity present in the data, allowing for good hyperplane separability in the lifted, higher-dimensional space.

> *Final Aside.* Kernel versions of many other algorithms (beyond SVM) exist as a way to extend linear methods to nonlinear data. We will mention some of these methods (e.g., PCA) towards the end of the course.